ESPECIFICAÇÃO TÉCNICA OFICIAL

All Tickets — Aplicação FullStack

v1.0.0 • Produção

1. Visão Geral e Stack Tecnológica

O **All Tickets** é uma solução FullStack para gerenciamento centralizado de eventos, comercialização de ingressos digitais e controle transacional de repasses financeiros compartilhados. O sistema oferece controle de acesso estrito baseado em funções (RBAC) para os perfis **PUBLICO**, **EMPRESA** e **ADMIN**, garantindo integridade e prevenindo fraudes na entrada dos eventos via validação de hashes criptográficos SHA-256.

Stack Tecnológica Homologada:

- **Backend:** Python 3.11+ com framework Django em arquitetura MVT (Model-View-Template).
- **Frontend:** Interface reativa em JavaScript ES6+ e componentes visualmente otimizados com CSS.
- **Persistência e ORM:** PostgreSQL 16+ com integridade referencial estrita e transações atômicas.
- **Segurança e Criptografia:** Hashes únicos SHA-256 (módulo `crypto.subtle` / `hashlib`) para emissão e validação de bilhetes.
- **Validação e Testes:** Pydantic v2 para DTOs e módulo `pytest / django.test` para testes automatizados BDD.

2. Arquitetura e Restrições de Domínio

A aplicação adota o padrão **Django MVT com camadas limpas de domínio**, isolando regras de negócio essenciais contra dependências externas:

- **Domain:** Entidades puras (`User`, `Event`, `Ticket`, `FinancialTransaction`) e Exceções de Domínio.
- **Use Cases:** Checkout com Split Financeiro (80/20), Validação de Portaria Antifraude e Aprovação de Parceria.
- **Controle de Acesso RBAC:** Bloqueio estrito de execução de rotas de acordo com a função autenticada.
  - `PUBLICO`: Navegação no feed, curtidas, checkout e visualização de ingressos em carteira.
  - `EMPRESA`: Painel do Produtor, cadastro de novos eventos e Módulo Validador de Portaria.
  - `ADMIN`: Painel de Métricas Globais, Auditoria de Solicitações e Promoção de Papéis.

3. Modelo de Dados (PostgreSQL 16+)

Tabelas principais de Usuários e Solicitações de Parceria:

```
  -- TB_USUARIOS: Registro central de contas e papéis RBAC
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(64) NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'PUBLICO', -- 'PUBLICO', 'EMPRESA', 'ADMIN'
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- TB_SOLICITACOES_VINCULACAO: Fila de auditoria empresarial
CREATE TABLE company_binding_requests (
```

3. Modelo de Dados (Continação DDL SQL)

```
  -- TB_EVENTOS: Maneamento de atrações publicadas
CREATE TABLE events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(120) NOT NULL,
    category VARCHAR(50) NOT NULL  -- 'Shows e Festivais', 'Teatro e Cultura', 'Festas e Nightlife'
    price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),
    stock INT NOT NULL CHECK (stock >= 0),
    likes_count INT DEFAULT 0
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- TB_INGRESSOS: Bilhetes emitidos com Hashes Criptográficos SHA-256
CREATE TABLE tickets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid()
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    event_id UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
    qr_hash VARCHAR(64) UNIQUE NOT NULL,
    is_used BOOLEAN NOT NULL DEFAULT FALSE
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    used_at TIMESTAMP WITH TIME ZONE
);

-- TB_FINANCEIRO: Livro caixa transacional e divisão 80/20
CREATE TABLE financial_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid()
    ticket_id UUID UNIQUE NOT NULL REFERENCES tickets(id) ON DELETE CASCADE,
    total_amount DECIMAL(10, 2) NOT NULL
    platform_share_20 DECIMAL(10, 2) NOT NULL,
    company_share_80 DECIMAL(10, 2) NOT NULL
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

80%	20%
Repasse Produtor (Empresa)	**Comissão Plataforma (All Tickets)**

4. Interface Gráfica WEB e Recursos Integrados

A interface do All Tickets fornece fluxos otimizados e seguros adaptados para cada nível de permissão:

- **Feed de Eventos Público (`/public`):** Catálogo com busca por texto, filtro de categorias oficiais (*Shows e Festivais*, *Teatro e Cultura*, *Festas e Nightlife*), contadores de curtidas e verificação de estoque em tempo real.
- **Carteira Digital de Ingressos:** Exibição dos bilhetes do usuário autenticado com seus respectivos Hashes SHA-256, modal de QR Code visual e status atualizado (`ATIVO` vs `UTILIZADO`).
- **Painel do Produtor (`/company`):** KPIs de vendas (Ingressos Vendidos, Receita Bruta, Saldo Líquido 80%), formulário de publicação de atrações e o **Módulo Validador de Portaria**.
- **Painel Administrativo (`/admin`):** Métricas financeiras globais (GMV e comissão 20%), Gestão de Usuários e Fila de Auditoria de Solicitações para promoção de contas `PUBLICO` em `EMPRESA`.

Funcionamento do Validador de Portaria (Antifraude):

A equipe da portaria digita ou lê o QR Hash do ingresso. O sistema consulta a tabela `tickets`; se `is_used`

5. Contratos de API REST (v1)

5.1 Checkout e Split Financeiro: `POST /api/v1/events/{event_id}/purchase`

```
  -- Response 201 Created
{
  "ticket": {
    "id": "8f3b20a1-9d11-4c62-81e3-2e008a129999",
    "event_title": "Maceió Summer Sounds 2026",
    "qr_hash": "QR-A9F82B10C4",
    "is_used": false
  }
  "financial_split": {
    "total_amount": 100.00
    "company_share_80": 80.00,
    "platform_share_20": 20.00
  }
}
```

5.2 Validação de Portaria Antifraude: `POST /api/v1/tickets/validate`

```
  -- Request: { "qr_hash": "QR-A9F82B10C4" }
-- Response 200 OK: { "status": "VALIDATED", "message": "Entrada liberada com sucesso!" }
-- Response 400 Bad Request: { "error": "ALERTA: Ingresso já utilizado anteriormente na portaria."
}
```

6. Regras de Negócio (RN) e Casos de Borda (CB)

- **RN-01 (Divisão Financeira 80/20):** Toda compra efetuada é dividida automaticamente em 80% para a empresa produtora e 20% retidos como comissão da plataforma.
- **RN-02 (Controle Rigoroso de Estoque):** A compra só é autorizada se `stock > 0`. A confirmação decrementa o estoque de forma atômica.
- **RN-03 (Unicidade Hash SHA-256):** Todo bilhete possui uma hash única e inviolável.
- **RN-04 (Antifraude / Trava de Reuso):** Um ingresso só pode ser validado uma única vez. Após a leitura, a flag `is_used` passa a ser `TRUE`.
- **RN-05 (Promoção ADM):** Apenas contas `ADMIN` podem aprovar solicitações e alterar papéis de usuários.
- **CB-01 / CB-02:** Tentativas de compra sem estoque retornam `400 Bad Request`. Tentativas de reutilizar bilhetes já lidos disparam alerta de fraude.

7. Critérios de Aceite (Testes BDD)

Cenário BDD 1: Validação de Portaria e Bloqueio Antifraude

Dado que existe um ingresso com `qr_hash = "QR-A9F82B10C4"` e `is_used = False`,
Quando a portaria envia requisição `POST /api/v1/tickets/validate`,
Então a resposta deve ser `200 OK` e a flag passa para `is_used = True`.
E quando for enviada uma segunda requisição utilizando o mesmo hash,
Então o status deve ser `400 Bad Request` com a mensagem: *"ALERTA: Ingresso já utilizado anteriormente na portaria"*.